Data preprocessing and preparation

Xuan–Hieu Phan

VNU University of Engineering and Technology
hieupx@vnu.edu.vn

Updated: September 11, 2023

Outline

1 Introduction

2 Data cleaning

3 Data integration

4 Data transformation

5 Data reduction

6 Data sampling

7 References and Summary

Outline

1 Introduction

2 Data cleaning

3 Data integration

4 Data transformation

5 Data reduction

6 Data sampling

7 References and Summary

Why data preprocessing?

- Real–world databases are highly **susceptible** to **noisy**, **missing**, and **inconsistent data** due to their typically huge size (often several gigabytes or more) and their likely origin from multiple, heterogenous sources.
- Original data are normally in **raw formats**, not ready for analysis and mining models.
- **Low–quality data** will lead to **low–quality mining results**.
- Data have quality if they satisfy the requirements of the intended use. There are many factors comprising **data quality**, including *accuracy*, *completeness*, *consistency*, *timeliness*, *believability*, and *interpretability*.
- How can the data be preprocessed in order to help **improve the quality of the data** and, consequently, of the mining results? How can the data be preprocessed so as to improve the efficiency and ease of the mining process?

Major tasks in data preprocessing

- **Data cleaning**:

  "clean" the data by filling in missing values, smoothing noisy data, identifying or removing outliers, and resolving inconsistencies.

- **Data integration**:

  involve smoothly merging data from multiple sources, e.g., databases, data cubes, or files into a coherent data store such as a data warehouse.

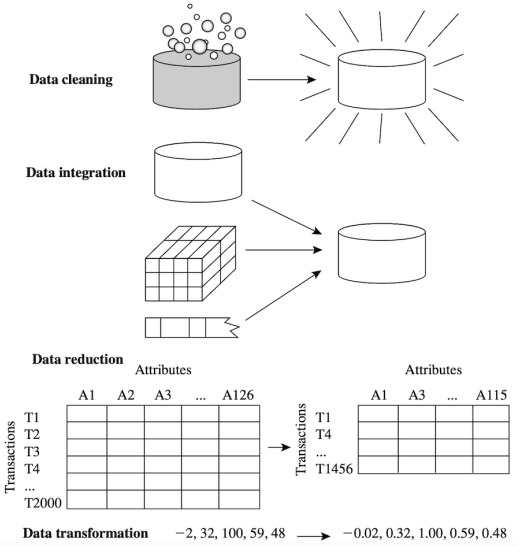
- **Data reduction**:

  reduce data in different ways, e.g., dimensionality reduction, removing irrelevant variables/attributes, data reduction using sampling, etc.

- **Data transformation**:

  perform data type conversion, discretization, data smoothing, data scaling and normalization, etc.

Major tasks in data preprocessing (cont'd)



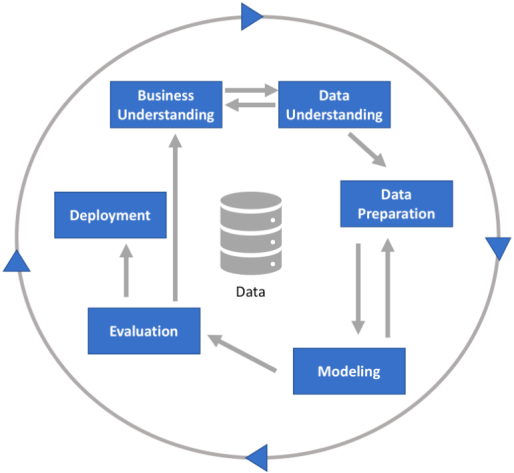
Cross–industry standard process for data mining

- The **cr**oss–**i**ndustry **s**tandard **p**rocess for **d**ata **m**ining (CRISP–DM) is a process model that serves as the base for a data science process. It has six sequential phases:

  1. Business understanding
  2. Data understanding
  3. Data preparation
  4. Modeling
  5. Evaluation
  6. Deployment

- Published in 1999 to standardize data mining processes across industries, it has since become the most common methodology for data mining, analytics, and data science projects.

CRISP–DM process



CRISP–DM phase 1: Business understanding

This phase focuses on understanding the objectives and requirements of the project. It includes four tasks:

1. **Determine business objectives**:

   thoroughly understand, from a business perspective, what the customer/company really wants to accomplish, and then define business success criteria.

2. **Assess situation**:

   determine resources availability, project requirements, assess risks and contingencies, and conduct a cost–benefit analysis.

3. **Determine data mining goals**:

   in addition to defining the business objectives, you should also define what success looks like from a technical data mining perspective.

4. **Produce project plan**:

   select technologies and tools and define detailed plans for each project phase.

CRISP–DM phase 2: Data understanding

This phase drives the focus to identify, collect, and analyze the data sets that can help you accomplish the project goals. This phase also has four tasks:

1. **Collect initial data**:

   acquire the necessary data and (if necessary) load it into your analysis tool.

2. **Describe data**:

   examine the data and document its surface properties like data format, number of records, or field identities.

3. **Explore data**:

   dig deeper into the data. Query it, visualize it, and identify relationships among the data.

4. **Verify data quality**:

   how clean/dirty is the data? Document any quality issues.

CRISP–DM phase 3: Data preparation

This phase prepares the final data for modeling. It has five tasks:

1. **Select data**:

   determine which data sets will be used and document reasons for inclusion/exclusion.

2. **Clean data**:

   often this is the lengthiest task. Without it, you will likely fall victim to garbage-in, garbage-out. A common practice during this task is to correct, impute, or remove erroneous values.

3. **Construct data**:

   derive new attributes that will be helpful. For example, derive someone's body mass index from height and weight fields.

4. **Integrate data**:

   create new data sets by combining data from multiple sources.

5. **Format data**:

   re-format data as necessary. For example, you might convert string values that store numbers to numeric values so that you can perform mathematical operations.

Outline

1 Introduction

2 Data cleaning

3 Data integration

4 Data transformation

5 Data reduction

6 Data sampling

7 References and Summary

Data cleaning

1. Handling missing values
2. Handling incorrect and inconsistent values
3. Handling noisy data

Handling missing values

1. Ignore tuples with missing values
2. Fill in the missing values manually
3. Use a global constant to fill in the missing values
4. Use a measure of central tendency for the attribute
5. Use the attribute mean or median for all samples belonging to the same class as the given tuple
6. Use the most probable value to fill in the missing values

Ignore tuples with missing values

- This is usually done when **the class label is missing** (assuming the mining task involves classification).
- This method is **not very effective**, unless the tuple contains several attributes with missing values.
- It is especially **poor** when the percentage of missing values per attribute varies considerably.
- By ignoring tuples, we do not make use of the remaining attributes' values in the tuples.

Fill in the missing values manually

- In general, this approach is time consuming and may **not be feasible** given a **large data** set with many missing values.

Use a global constant to fill in the missing values

- Replace all missing attribute values by **the same constant** such as a label like "*Unknown*" or $-\infty$.
- If missing values are replaced by, say, "*Unknown*" then the mining program may **mistakenly think** that they form an interesting concept, since they all have a value in common – that of "*Unknown*".
- Hence, although this method is simple, it is **not reliable**.

Use a measure of central tendency for the attribute

- Use a measure of **central tendency** for the attribute (e.g., the **mean** or **median**) to fill in the missing values.
- For normal (symmetric) data distributions, the mean can be used, while **skewed** data distribution should employ the **median**.

Class–based filling

- Use the **attribute mean** or **median** for all samples belonging to the **same class** as the given tuple.
- For example, if classifying customers according to `credit_risk`, we may replace the missing value with the mean `income` value for customers in the same credit risk category as that of the given tuple.
- If the data distribution for a given class is **skewed**, the **median** value is a better choice

Use the most probable value to fill in the missing values

- This may be determined with **regression**, **inference-based tools** using a Bayesian formalism, or decision tree induction.
- For example, using the other customer attributes in your data set, you may construct a decision tree to **predict the missing values** for `income`.
- This is a **popular strategy**. In comparison to the other methods, it uses the most information from the **present data** to predict missing values. By considering the other attributes' values in its estimation of the missing value for `income`, there is a greater chance that the relationships between `income` and the other attributes are preserved.

Missing values may not errors in data

- It is important to note that, in some cases, a **missing value** may **not imply an error** in the data!
- For example, when applying for a credit card, candidates may be asked to supply their driver's license number. Candidates who **do not have a driver's license** may naturally **leave this field blank**.
- Ideally, each attribute should have one or more **rules** regarding the **null condition**. The rules may specify whether or not nulls are allowed and/or how such values should be handled or transformed.
- Fields may also be **intentionally left blank** if they are to be provided in a later step of the business process. Hence, although we can try our best to clean the data after it is seized, good database and data entry procedure design should help minimize the number of missing values or errors in the first place.

Handling incorrect and inconsistent values

The key methods that are used for removing or correcting the incorrect and inconsistent entries are as follows:

- Inconsistency detection
- Domain knowledge
- Data-centric methods

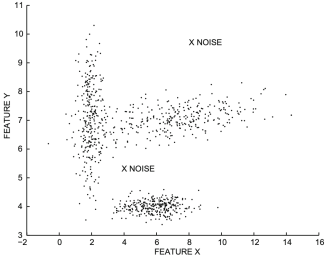
Inconsistency detection

- This is typically done when the data is available from **different sources** in **different formats**.
- For example, **a person's name** may be spelled out in **full** in one source, whereas the other source may only contain the **initials** and a **last name**.
- In such cases, the key issues are **duplicate detection** and **inconsistency detection**.
- These topics are studied under the general umbrella of **data integration** within the database field.

Domain knowledge

- A significant amount of **domain knowledge** is often available in terms of the ranges of the attributes or rules that specify the relationships across different attributes.
- For example, if the country field is “United States”, then the city field cannot be “Shanghai”.
- Many data scrubbing and data auditing tools have been developed that use such domain knowledge and constraints to detect incorrect entries.

Data–centric methods



- In these cases, the **statistical behavior** of the data is used to **detect outliers**. For example, the two isolated data points in the above figure marked as "noise" are outliers. These isolated points might have arisen because of errors in the data collection process.
- However, this may **not always** be the case because **the anomalies** may be **the result of interesting behavior** of the underlying system. Therefore, any detected outlier may need to be manually examined before it is discarded. The use of data–centric methods for cleaning can sometimes be **dangerous** because they can result in the **removal of useful knowledge** from the underlying system.

Handling noisy data

What is noise? Noise is a **random error or variance** in a measured variable. In the last lecture (data understanding) we saw some techniques like **five–number summary** and **boxplots** to identify noise or outliers. The following are some smoothing techniques to deal with noisy data:

- Binning
- Regression
- Outlier analysis

Handling noisy data: binning

- Binning methods smooth a sorted data value by consulting its "neighborhood," that is, the values around it.
- The sorted values are put into a number of "buckets," or *bins*.
- Because binning methods consult the neighborhood of values, they perform *local* smoothing.
- Example (next slide): in **smoothing by bin means**, each value in a bin is replaced by the mean value of the bin. For example, the mean of the values 4, 8, and 15 in Bin 1 is 9. Therefore, each original value in this bin is replaced by the value 9.
- Similarly, **smoothing by bin medians** can be employed, in which each bin value is replaced by the bin median.
- In **smoothing by bin boundaries**, the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value. In general, the larger the width, the greater the effect of the smoothing.

Handling noisy data: binning example

Sorted data for *price* (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15
Bin 2: 21, 21, 24
Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9
Bin 2: 22, 22, 22
Bin 3: 29, 29, 29

Smoothing by bin boundaries:

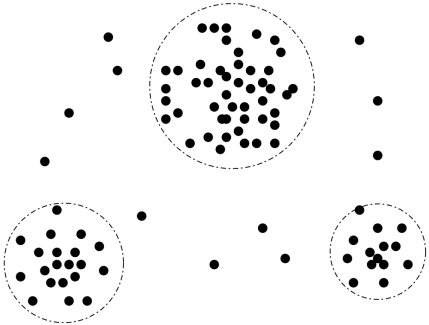
Bin 1: 4, 4, 15
Bin 2: 21, 21, 24
Bin 3: 25, 25, 34

Binning methods for data smoothing [1]

Handling noisy data: regression

- Data smoothing can also be done by regression, a technique that conforms data values to a function.
- *Linear regression* involves finding the "best" line to fit two attributes (or variables) so that one attribute can be used to predict the other.
- *Multiple linear regression* is an extension of linear regression, where more than two attributes are involved and the data are fit to a multidimensional surface.

Handling noisy data: outlier analysis



Removing noisy data (outliers) using clustering [1]

- Outliers may be detected by clustering, for example, where similar values are organized into groups, or "clusters."
- Intuitively, values that fall outside of the set of clusters may be considered outliers.

Outline

1 Introduction

2 Data cleaning

3 Data integration

4 Data transformation

5 Data reduction

6 Data sampling

7 References and Summary

Data integration

- Data mining often requires **data integration**.
- Data integration:

  the **merging** of data from **multiple** data sources.
- Careful data integration:

  → reduce and avoid **redundancies** and **inconsistencies**.

  → improve the **accuracy** and **speed** of the subsequent data mining process.
- The major tasks in data integration:
  1. Entity identification problem
  2. Redundancy and correlation analysis
  3. Tuple or record duplication
  4. Data value conflict detection and resolution

Entity identification problem

- *Schema integration* and *object matching* can be tricky.
  - How can equivalent real–world entities from multiple data sources be matched up?
  - This is referred to as the **entity identification problem**.
- Example:

  `customer_id` and `cust_number` in two databases refer to the same attribute?
- **Meta–data** can be used for schema matching.
- Meta–data of a data attribute may include:
  - Name of the attribute
  - Meaning or description of the attribute
  - Data type and range of values
  - Null rules for handling blank, zero, or null value

Redundancy and correlation analysis

- **Redundancy** is another important issue in data integration.
- Example:

  `annual_revenue` may be redundant if it can be "derived" from other attributes.
- Inconsistent attribute naming can also cause redundancies in the resulting data.
- Some redundancies can be detected by **correlation analysis**.
- Correlation analysis can measure **how strongly** one attribute **implies** the other.
- Correlation analysis for nominal/categorical data:

  use the χ^2 (chi–square) test of independence (previous lecture: *Data understanding*).
- Correlation analysis for numeric attributes:

  use covariance or correlation coefficient, e.g., Pearson correlation (previous lecture).

Tuple duplication

- Duplication should also be detected at the tuple level.

  e.g., where there are two or more identical tuples for a given unique data entry case.

- Inconsistencies often arise between various duplicates

  due to inaccurate data entry or updating some but not all data occurrences.

- Example:

  if a purchase order database contains attributes for the purchaser's name and address instead of **a key** to this information in a purchaser database, **discrepancies** can occur, such as the **same purchaser's name** appearing with **different addresses** within the purchase order database.

Data value conflict detection and resolution

- Data integration also involves the detection and resolution of data value conflicts. For example, for the same real–world entity, attribute values from different sources may differ.
- This may be due to differences in representation, scaling, or encoding. For instance, a weight attribute may be stored in metric units in one system and British imperial units in another. For a hotel chain, the price of rooms in different cities may involve not only different currencies but also different services (e.g., free breakfast) and taxes.
- When exchanging information between schools, for example, each school may have its own curriculum and grading scheme. One university may adopt a quarter system, offer three courses on database systems, and assign grades from A+ to F, whereas another may adopt a semester system, offer two courses on databases, and assign grades from 1 to 10.

Outline

1 Introduction

2 Data cleaning

3 Data integration

4 Data transformation

5 Data reduction

6 Data sampling

7 References and Summary

Data transformation

1. Data conversion and discretization
2. Data smoothing
3. Data aggregation
4. Construction of attributes
5. Data scaling and normalization
6. Concept hierarchy generation for nominal data

Data conversion and discretization

- Numeric to categorical data: discretization
- Categorical to numeric data: binarization
- Text to numeric data
- Time–series to discrete sequence data

Numeric to categorical data: discretization

- Discretization is the task that divides the value range of a numeric attribute into k sub–ranges.
- Then, the attribute is assumed to contain k different categorical labeled values from 1 to k, depending on the range in which the original attribute lies.
- For example, consider the age attribute. One could create sub–ranges $[0, 10]$, $[11, 20]$, $[21, 30]$, and so on. The *symbolic* value for any record in the sub–range $[11, 20]$ is "2" and the symbolic value for a record in the sub–range $[21, 30]$ is "3". Because these are symbolic values, no ordering is assumed between the values "2" and "3".
- Variations within a sub–range are not distinguishable after discretization. Thus, the discretization process does lose some information for the mining process.
- However, for some applications, this loss of information is not a big problem.

Discretization challenges and techniques

- One challenge with discretization is that the data may be non–uniformly distributed across the different intervals.
- For example, for the *salary* attribute, a large subset of the population may be grouped in the $[40,000,\ 80,000]$ sub–range, but very few will be grouped in the $[1,040,000,\ 1,080,000]$ sub–range (both sub–ranges have the same size).
- Thus, the use of sub–ranges of equal size may not be very helpful in discriminating between different data segments. Many attributes, such as age, are not as non–uniformly distributed, and therefore ranges of equal size may work reasonably well.
- The discretization process can be performed in a variety of ways depending on application–specific goals:
  - **Equi–width sub–ranges**
  - **Equi–log sub–ranges**
  - **Equi–depth sub–ranges**
  - **Clustering–based discretization**

Discretization: equi–width sub–ranges

- In this case, each sub–range $[a, b]$ is chosen in such a way that $b - a$ is the same for each sub–range.
- This approach has the drawback that it will not work for data sets that are distributed non–uniformly across the different sub–ranges.
- To determine the actual values of the sub–ranges, the minimum and maximum values of each attribute are determined. This range $[min, max]$ is then divided into k sub–ranges of equal length.

Discretization: equi–log sub–ranges

- Each sub–range $[a, b]$ is chosen in such a way that $\log(b) - \log(a)$ has the same value.
- This kinds of sub–range selection has the effect of geometrically increasing sub–ranges $[a, a\alpha]$, $[a\alpha, a\alpha^2]$, $[a\alpha^2, a\alpha^3]$, and so on, for some $\alpha > 1$.
- This kind of sub–range may be useful when the attribute shows an exponential distribution across a range.
- In fact, if the attribute frequency distribution for an attribute can be modeled in functional form, then a natural approach would be to select sub–ranges $[a, b]$ such that $f(b) - f(a)$ is the same for some function $f(\cdot)$.
- The idea is to select this function $f(\cdot)$ in such a way that each sub–range contains an approximately similar number of records. However, in most cases, it is hard to find such a function $f(\cdot)$ in closed form.

Discretization: equi–depth sub–ranges

- In this case, the sub–ranges are selected so that each sub–range has an equal number of records.
- The idea is to provide the same level of granularity to each sub–range.
- An attribute can be divided into equi–depth sub–ranges by first sorting it, and then selecting the division points on the sorted attribute value, such that each sub–range contains an equal number of records.

Clustering–based discretization

- Cluster analysis is a popular data discretization method.
- A clustering algorithm can be applied to discretize a numeric attribute, X, by partitioning the values of X into clusters or groups.
- Clustering takes the distribution of X into consideration, as well as the closeness of data points, and therefore is able to produce high–quality discretization results.

Categorical to numeric data: binarization

- In some cases, it is desirable to use numeric data mining algorithms on categorical data.
- Because binary data is a special form of both numeric and categorical data, it is possible to convert the categorical attributes to binary form and then use numeric algorithms on the binarized data.
- If a categorical attribute has k different values, then k different binary attributes are created. Each binary attribute corresponds to one possible value of the categorical attribute.
- Therefore, exactly one of the k attributes takes on the value of 1, and the remaining take on the value of 0.

ID	Male	Female	IsStudent	IsEmployee	IsRetired
001	1	0	1	0	0
002	0	1	1	0	0
003	1	0	0	1	0
004	1	0	0	0	1
005	0	1	0	1	0

Text to numeric data

- Vector space model (a.k.a. bag of words): converting documents, paragraphs, sentences into sparse vectors. The vector dimension is the size of vocabulary of the text collection. Weights in vectors can be binary, raw frequency, normalized frequency, or TF–IDF.
- N–gram model: this is a special case of vector space model where consecutive words or tokens can be combine to form n–grams (uni–grams, bi–grams, tri–grams, etc.)
- Latent semantic and topics models: transforming documents, paragraphs, sentences into dense semantic/topic vectors. The dimension is much smaller, e.g., 50, 100, 200, 300. Well–known models are LSA, LDA, and various topic models.
- Text embeddings: using deep representation learning models to transform texts (words, sentences, paragraphs, documents) into embeddings (dense) vectors. Well–known techniques are Word2Vec, GloVe, Doc2Vec, etc.
- Popular libraries: NLTK, Gensim, CoreNLP, and spaCy. For Vietnamese NLP: VLSP libraries, VnCoreNLP, PhoBERT, and many more from https://github.com/topics/vietnamese-nlp.

Time–series to discrete sequence data

Converting time–series data to discrete sequence using *symbolic aggregate approximation* (SAX). This method comprises two steps:

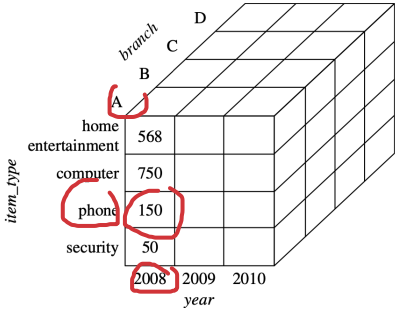
- *Window–based averaging*: the series is divided into windows of length w, and the average time–series value over each window is computed.
- *Value–based discretization*: the averaged time–series values are discretized into a smaller number of equi–depth intervals.
  - The idea is to ensure that each symbol has an approximately equal frequency in the time–series. This is identical to the equi–depth discretization of numeric attributes that was discussed earlier.
  - The interval boundaries are constructed by assuming that the time–series values are distributed with a Gaussian assumption. The mean and standard deviation of the (windowed) time–series values are estimated in the data–driven manner.
  - The quantiles of the Gaussian distribution are used to determine the boundaries of the intervals. This is more efficient than sorting all the data values to determine quantiles, and it may be a more practical approach for a long time–series. The number of intervals is normally from 3 to 10.

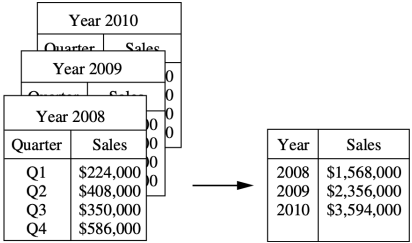
Data smoothing

- **Smoothing**, which works to remove noise from data. Techniques include *binning*, *regression*, and *clustering*.
- Data smoothing has been presented in the Data cleaning section (subsection: Handling noisy data).

Data aggregation

- **Aggregation**, where summary or aggregation operations are applied to the data. This step is typically used in constructing a data cube for data analysis at multiple levels of abstraction.
- Dat cube is a common concept in data warehousing that represents a multidimensional dataset where each attribute may have a concept hierarchy. For example, attribute `Time` can be at different levels of granularity like $\{year > quarter > month > week > day > hour\}$.





Year 2010

Year 2009

Year 2008

Quarter	Sales
Q1	$224,000
Q2	$408,000
Q3	$350,000
Q4	$586,000

Year	Sales
2008	$1,568,000
2009	$2,356,000
2010	$3,594,000

Construction of attributes

- In attribute construction or feature construction, new attributes are constructed and added to help the mining process, e.g., improving the accuracy and understanding of structure in high–dimensional data.
- For example, we may wish to add the attribute `area` based on the attributes `height` and `width`.
- New attributes can also help data miners and analysts to explore, observe, and understand the data more easily.

Data scaling and normalization

- In many cases, the different features represent different scales of reference and may therefore not be comparable to one another.
- For example, an attribute such as age is drawn on a very different scale than an attribute such as salary. The latter attribute is typically orders of magnitude larger than the former. As a result, any aggregate function computed on the different features (e.g., Euclidean distances) will be dominated by the attribute of larger magnitude.
- To address this problem, values of attributes are usually re–scaled or normalized to a more suitable range.
- There are two main standardization techniques:
  - **Min-max normalization**
  - **Standard score normalization**
  - **Decimal scaling**

Min–max normalization

- Let $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$ be a univariate data sample consisting of n observations/values drawn from a variable/attribute X. In *min–max normalization*, each value is scaled as follows:

$$x'_i = \frac{x_i - min_X}{max_X - min_X}(max_X^{new} - min_X^{new}) + min_X^{new} \quad (1)$$

  where:

  - min_X and max_X are the minimum and the maximum values in $\mathcal{D}$ (i.e., before normalization), respectively.
  - min_X^{new} and max_X^{new} are the minimum and the maximum values of the data attribute after being normalized, respectively.

- After transformation, all the new values x'_i $(i = 1..n)$ is in $[min_X^{new}, max_X^{new}]$. Normally, $min_X^{new} = 0$ and $max_X^{new} = 1$ and the new range is $[0, 1]$.

- Min–max normalization is also called **range normalization**. This normalization technique can also be applied to any numeric data attributes in bivariate and multivariate data.

Standard score normalization

- Let $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$ be a univariate data sample consisting of n observations/values drawn from a variable/attribute X. In standard score normalization, also called z–normalization, each value is replaced by its z-score as follows:

$$x'_i = \frac{x_i - \hat{\mu}}{\hat{\sigma}} \tag{2}$$

  where:
  - $\hat{\mu}$ is the sample mean.
  - $\hat{\sigma}$ is the sample standard deviation.

- After transformation, the new attribute has mean $\hat{\mu}' = 0$, and standard deviation $\hat{\sigma}' = 1$.
- The vast majority of the normalized values will typically lie in the range $[-3, 3]$ under the normal distribution assumption.

Decimal scaling

- Normalization by decimal scaling normalizes by moving the decimal point of values of a numeric attribute X.
- The number of decimal points moved depends on the maximum absolute value of X.
- A value, x_i, of X is normalized to x'_i by computing:

$$x'_i = \frac{x_i}{10^j} \tag{3}$$

  where j is the smallest integer such that $max(|x'_i|) < 1$.
- Example: suppose that the recorded values of X range from 986 to 917. The maximum absolute value of X is 986. To normalize by decimal scaling, we therefore divide each value by 1000 (i.e., $j = 3$) so that 986 normalizes to 0.986 and 917 normalizes to 0.917.

Bivariate data sample

$$\mathcal{D} = \left(\begin{array}{ccc} \mathbf{x}_i & \textit{Age } (X_1) & \textit{Income } (X_2) \\ \hline \mathbf{x}_1 & 12 & 300 \\ \mathbf{x}_2 & 14 & 500 \\ \mathbf{x}_3 & 18 & 1000 \\ \mathbf{x}_4 & 23 & 2000 \\ \mathbf{x}_5 & 27 & 3500 \\ \mathbf{x}_6 & 28 & 4000 \\ \mathbf{x}_7 & 34 & 4300 \\ \mathbf{x}_8 & 37 & 6000 \\ \mathbf{x}_9 & 39 & 2500 \\ \mathbf{x}_{10} & 40 & 2700 \end{array}\right)$$

Let $\mathcal{D}$ be a bivariate data sample consisting of 10 observations/points drawn from two variables/attributes: Age (X_1) and Income (X_2) [4].

Min–max normalization example

- $max_{Age} = 40$ and $min_{Age} = 12$. For example, $x_{31} = 18$, after normalization, $x'_{31} = \frac{18-12}{40-12} = 0.214$.
- Similarly, $max_{Income} = 6000$ and $min_{Income} = 300$.
- After normalization, we have the new data sample $\mathcal{D}'$ with two new attributes Age' and $Income'$ as follows:

$$\mathcal{D}' = \left(\begin{array}{ccc} \mathbf{x}_i & Age' \ (X'_1) & Income' \ (X'_2) \\ \hline \mathbf{x}_1 & 0.0 & 0.0 \\ \mathbf{x}_2 & 0.071 & 0.035 \\ \mathbf{x}_3 & 0.214 & 0.123 \\ \mathbf{x}_4 & 0.393 & 0.298 \\ \mathbf{x}_5 & 0.536 & 0.561 \\ \mathbf{x}_6 & 0.571 & 0.649 \\ \mathbf{x}_7 & 0.786 & 0.702 \\ \mathbf{x}_8 & 0.893 & 1.0 \\ \mathbf{x}_9 & 0.964 & 0.386 \\ \mathbf{x}_{10} & 1.0 & 0.421 \end{array}\right)$$

Standard score normalization example

- For z–normalization, we first compute the mean and standard deviation of both attributes:

$$\hat{\mu}_{Age} = 27.2 \text{ and } \hat{\sigma}_{Age} = 9.77$$
$$\hat{\mu}_{Income} = 2680 \text{ and } \hat{\sigma}_{Income} = 1726.15$$

- The new data after z–normalization is as follows:

$$\mathcal{D}' = \left(\begin{array}{ccc} \mathbf{x}_i & Age' \ (X_1') & Income' \ (X_2') \\ \hline \mathbf{x}_1 & -1.56 & -1.38 \\ \mathbf{x}_2 & -1.35 & -1.26 \\ \mathbf{x}_3 & -0.94 & -0.97 \\ \mathbf{x}_4 & -0.43 & -0.39 \\ \mathbf{x}_5 & -0.02 & 0.48 \\ \mathbf{x}_6 & 0.08 & 0.76 \\ \mathbf{x}_7 & 0.70 & 0.94 \\ \mathbf{x}_8 & 1.0 & 1.92 \\ \mathbf{x}_9 & 1.21 & -0.10 \\ \mathbf{x}_{10} & 1.31 & 0.01 \end{array} \right)$$

Distance between points before and after normalization

- The Euclidean distance between $\mathbf{x}_1$ and $\mathbf{x}_2$ according to the original data:

$$\begin{aligned}\|\mathbf{x}_1 - \mathbf{x}_2\| &= \|(-2, -200)^T\| \\ &= \sqrt{(-2)^2 + (-200)^2} = \sqrt{40004} = \mathbf{200.01}\end{aligned}$$

- According to the data after min–max normalization:

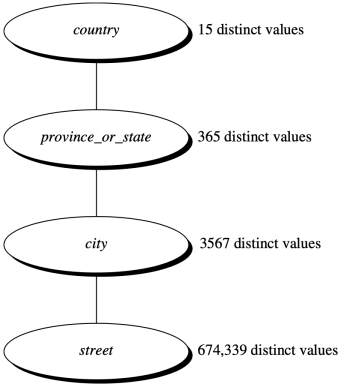
$$\begin{aligned}\|\mathbf{x}'_1 - \mathbf{x}'_2\| &= \|(0, 0)^T - (0.071, 0.035)^T\| \\ &= \|(-0.071, -0.035)^T\| = \mathbf{0.079}\end{aligned}$$

- According to the data after z–normalization:

$$\begin{aligned}\|\mathbf{x}'_1 - \mathbf{x}'_2\| &= \|(-1.56, -1.38)^T - (1.35, -1.26)^T\| \\ &= \|(-0.21, -0.12)^T\| = \mathbf{0.242}\end{aligned}$$

Concept hierarchy generation for nominal data

Concept hierarchy generation for nominal data, where attributes such as *street* can be generalized to higher–level concepts, like *city* or *country*. Many hierarchies for nominal attributes are implicit within the database schema and can be automatically defined at the schema definition level.



Example of concept hierarchy [1]

Outline

1 Introduction

2 Data cleaning

3 Data integration

4 Data transformation

5 Data reduction

6 Data sampling

7 References and Summary

Overview of data reduction

- Data reduction techniques can be applied to obtain a reduced representation of the data set that is much smaller in volume, yet closely maintains the integrity of the original data.
- Mining on the reduced data set should be more efficient yet produce the same (or almost the same) analytical results.
- **Dimensionality reduction** is the process of reducing the number of random variables or attributes under consideration. Dimensionality reduction methods include *wavelet transforms* and *principal component analysis*, which transform the original data onto a smaller space. *Attribute subset selection* identifies and remove irrelevant, weakly relevant, or redundant attributes.
- **Numerosity reduction** techniques replace the original data volume by alternative, smaller forms of data representation. Techniques are both *parametric* methods (*regression*, *log–linear models*) and *nonparametric* methods (*histograms*, *clustering*, *sampling*, *data cube aggregation*).

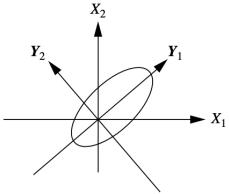
Data reduction methods

1. Principal component analysis (PCA)
2. Attribute subset selection
3. Histograms
4. Clustering
5. Data sampling
6. Data cube aggregation

Principal component analysis (PCA)

- Principal component analysis (PCA) is a method of dimensionality reduction.
- Suppose that the data to be reduced consist of m tuples or data vectors described by n attributes or dimensions.
- PCA searches for k n–dimensional orthogonal vectors that can best be used to represent the data, where $k \leq n$. The original data are thus projected onto a much smaller space, resulting in dimensionality reduction.
- PCA "combines" the essence of attributes by creating an alternative, smaller set of variables. The initial data can then be projected onto this smaller set.
- PCA often reveals relationships that were not previously suspected and thereby allows interpretations that would not ordinarily result.

Steps of PCA



PCA: Y_1 and Y_2 are the first two principal components for the given data. [1]

The steps of PCA are as follows:

1. The input data are normalized, so that each attribute falls within the same range. This step helps ensure that attributes with large domains will not dominate attributes with smaller domains.
2. PCA computes k orthonormal vectors that provide a basis for the normalized input data. These are *unit vectors* that each point in a direction perpendicular to the others. These vectors are referred to as the *principal components*. The input data are a linear combination of the principal components.

Steps of PCA (cont'd)

3. The principal components are sorted in order of decreasing "significance" or strength. The principal components essentially serve as a new set of axes for the data, providing important information about variance. That is, the sorted axes are such that the first axis shows the most variance among the data, the second axis shows the next highest variance, and so on. For example, the previous figure shows the first two principal components, Y_1 and Y_2, for the given set of data originally mapped to the axes X_1 and X_2. This information helps identify groups or patterns within the data.

4. Because the components are sorted in decreasing order of "significance," the data size can be reduced by eliminating the weaker components, that is, those with low variance. Using the strongest principal components, it should be possible to reconstruct a good approximation of the original data.

PCA can handle sparse data and skewed data. Multidimensional data can be handled by reducing to two dimensions. Principal components may be used as inputs to regression and cluster analysis.

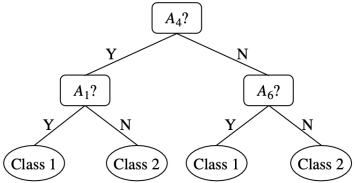
Attribute subset selection

- Data sets for analysis may contain hundreds of attributes, many of which may be irrelevant to the mining task or redundant.
- For example, if the task is to classify customers based on whether or not they are likely to purchase a popular new CD at `AllElectronics` when notified of a sale, attributes such as the customer's `telephone_number` are likely to be irrelevant, unlike attributes such as `age` or `music_taste`.
- Although it may be possible for a domain expert to pick out some of the useful attributes, this can be a difficult and time–consuming task, especially when the data's behavior is not well known.
- Leaving out relevant attributes or keeping irrelevant attributes may be detrimental, causing confusion for the mining algorithm employed. This can result in discovered patterns of poor quality.
- In addition, the added volume of irrelevant or redundant attributes can slow down the mining process.

Attribute subset selection (cont'd)

- Attribute subset selection reduces the data set size by removing irrelevant or redundant attributes (or dimensions).
- The goal of attribute subset selection is to find a minimum set of attributes such that the resulting probability distribution of the data classes is as close as possible to the original distribution obtained using all attributes.
- Mining on a reduced set of attributes has an additional benefit: It reduces the number of attributes appearing in the discovered patterns, helping to make the patterns easier to understand.

How to find a "*good*" subset of the original attributes?

Forward selection	Backward elimination	Decision tree induction
Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$ Initial reduced set: $\{\}$ => $\{A_1\}$ => $\{A_1, A_4\}$ => Reduced attribute set: $\{A_1, A_4, A_6\}$	Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$ => $\{A_1, A_3, A_4, A_5, A_6\}$ => $\{A_1, A_4, A_5, A_6\}$ => Reduced attribute set: $\{A_1, A_4, A_6\}$	Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$  => Reduced attribute set: $\{A_1, A_4, A_6\}$

Greedy (heuristic) methods for attribute subset selection. [1]

- For n attributes, there are 2^n possible subsets. An exhaustive search for the optimal subset can be prohibitively expensive. **Greedy** approach is normally used.
- The "best" (and "worst") attributes are typically determined using statistical tests. Many other attributes need an another measure like *information gain* (used in decision trees).

Attribute subset selection methods

1. **Stepwise forward selection**: the procedure starts with an empty reduced set of attributes. The best of the original attributes is determined and added to the reduced set, and similarly for the second, third steps, etc.
2. **Stepwise backward elimination**: the procedure starts with the full set of attributes. At each step, it removes the worst attribute remaining in the set.
3. **Combination of forward selection and backward elimination**: the stepwise forward selection and backward elimination methods can be combined in each step.
4. **Decision tree induction**: decision tree algorithms (e.g., ID3, C4.5, and CART) were originally intended for classification. All attributes that appear in the tree are assumed to be relevant.

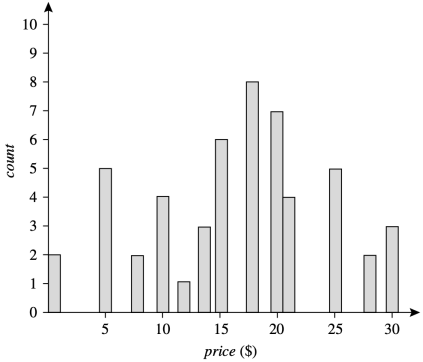
The stopping criteria for the methods may vary. The procedure may employ a threshold on the measure used to determine when to stop the attribute selection process.

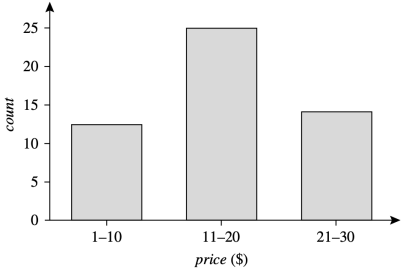
Histograms

- Histograms use binning to approximate data distributions and are a popular form of data reduction.
- A histogram for an attribute, X, partitions the data distribution of X into disjoint subsets, referred to as *buckets* or *bins*.
- If each bucket represents only a single attribute–value/frequency pair, the buckets are called *singleton buckets*. Often, buckets instead represent continuous ranges for the given attribute.
- How are the buckets determined and the attribute values partitioned? There are several partitioning ways:
  - **Equal–width**: in an equal–width histogram, the width of each bucket range is uniform.
  - **Equal–frequency** (or equal–depth): in an equal–frequency histogram, each bucket contains roughly the same number of contiguous data samples.
- Histograms are highly effective at approximating both sparse and dense data, as well as highly skewed and uniform data. The histograms can be extended for multiple attributes.

Example of histograms

The following data are a list of `AllElectronics` prices for commonly sold items [1]. The numbers have been sorted: {1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10, 12, 14, 14, 14, 15, 15, 15, 15, 15, 15, 18, 18, 18, 18, 18, 18, 18, 18, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 25, 25, 25, 25, 25, 28, 28, 30, 30, 30}.





Clustering

- Clustering techniques partition data objects into groups, or *clusters*, so that objects within a cluster are "similar" to one another and "dissimilar" to objects in other clusters.
- Similarity is commonly defined in terms of how "close" the objects are in space, based on a distance function.
- In data reduction, the cluster representations of the data are used to replace the actual data. The effectiveness of this technique depends on the data's nature. It is much more effective for data that can be organized into distinct clusters than for smeared data.
- Different clustering approaches and techniques will be given later in lecture **Data clustering**.

Data sampling

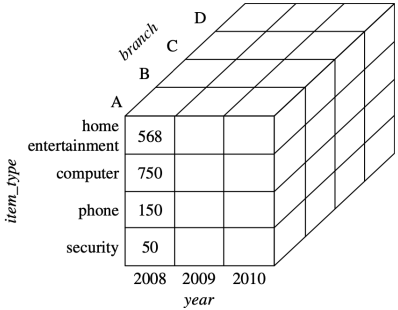
Data sampling can be used as a data reduction method because it allows a large data set to be represented by a much smaller random/non–random data sample (a subset). There are different sampling techniques and they are classified into two main types:

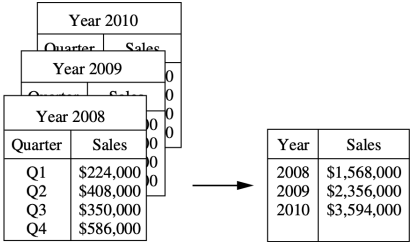
- Random sampling:
  - Simple random sampling (with and without replacement)
  - Stratified sampling
  - Cluster sampling
  - Systematic sampling
  - Multi–stage sampling
- Non–random sampling:
  - Convenience sampling
  - Judgement sampling
  - Quota sampling
  - Snowball sampling

Sampling techniques will be presented in details in the next section.

Data cube aggregation

- Dat cube is a common concept in data warehousing. A data cube represents a multidimensional dataset where each attribute may have a concept hierarchy. For example, attribute `Time` can be at different levels of granularity like $\{year > quarter > month > week > day > hour\}$.
- **Drill–down** allows to view data in more details while **roll–up** allows to view data in more general/summary.





Outline

1 Introduction

2 Data cleaning

3 Data integration

4 Data transformation

5 Data reduction

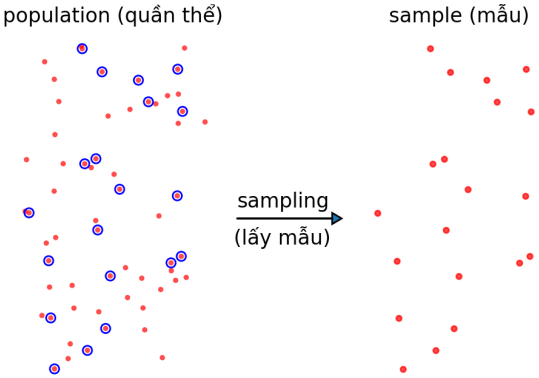
6 Data sampling

7 References and Summary

Data sampling

- Data sampling is the process of selecting a subset of data observations (data objects elements, instances, points), called a data *sample*, from a large *population* of data.
- The resulting data sample normally consists of data points that are good representatives of the population, reflecting the nature and characteristics of the population properly.
- Data sampling is critical because most of data analysis tasks will be performed on a data sample rather than on the whole population.
- Data sampling can be seen as a data reduction technique because it allows a large data set to be represented by a much smaller (random/non–random) data sample.
- Data sampling techniques are classified into two main types:
  1. Random sampling
  2. Non–random sampling

Data sampling (cont'd)



Random sampling methods

In random sapling, every data element in the population has a chance to be chosen via a random selection process. Popular random sampling techniques are as follows:

1. Simple random sampling
2. Stratified sampling
3. Cluster sampling
4. Systematic sampling
5. Multi–stage sampling

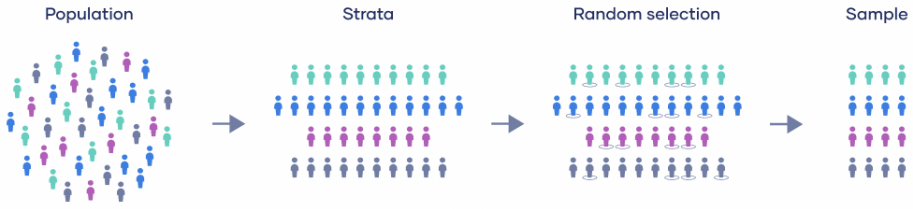
Simple random sampling

- A simple random sample is a subset of a statistical population in which each member of the subset has an equal probability of being chosen. It means that there is no bias when sampling.
  - A simple random sample takes a small, random portion of the entire population to represent the entire data;
  - Simple random sampling can be done using methods like lotteries or random draws;
  - This sampling technique is often used when we have little or no information about the population.
  - Simple random sampling can be *without replacement* or *with replacement*.
- Example:
  - Select randomly 30 employees from all 100 employees of a company (without replacement).
  - Select randomly 10000 documents from a collection of 100000 text documents (with replacement).

Bootstrap resampling

- Let $\mathcal{D} = \{x_1, x_2, \ldots, x_n\}$ be a dataset of n data element.
- Create a dataset $\mathcal{D}'$ by repeating n times of simple random sampling with replacement.
- Because sampling with replacement, the probability of being chosen of each data element is $\frac{1}{n}$, and the probability of NOT being chosen is $1 - \frac{1}{n}$.
- The probability of a data element $x_i \in \mathcal{D}$ and not being chosen to $\mathcal{D}'$ after repeating sampling with replacement n times is $P(x_i \notin \mathcal{D}') = (1 - \frac{1}{n})^n \approx e^{-1} = 0.368$.
- Then, the probability of a data element $x_i \in \mathcal{D}$ and also being chosen to $\mathcal{D}'$ after repeating the random sampling n times is $P(x_i \in \mathcal{D}') = 1 - P(x_i \notin \mathcal{D}') = 1 - 0.368 = 0.632$.
- Performing this sampling technique k times to get $\mathcal{D}_1$, $\mathcal{D}_2$, $\ldots$, $\mathcal{D}_k$ is called *bootstrap resampling*. This is commonly used in statistics, data analysis, and machine learning.

Stratified sampling

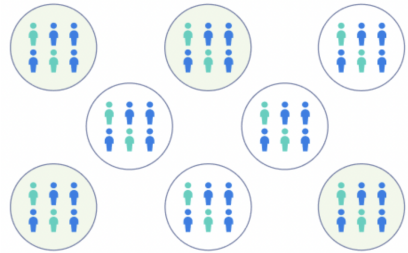


- In this technique, the data elements are first divided into different groups or strata. Elements in each group/stratum are similar in some way, and different from the elements in other strata.
- After division, the simple random sampling will be applied for each group/stratum to choose random elements into the final data sample.
- With this technique, we need an understanding of the population in order to divide the elements into strata.

Stratified sampling example

- The previous example: select randomly 30 employees from all 100 employees of a company.
- Now we have constraint that half of 30 selected employees are female. The simple random sampling will not always meet this constraint.
- Supposing that, in 100 employees of the company, there are 25 female. Then, we apply stratified random sampling by dividing the 100 employees into two groups/strata: 75 male and 25 female.
- Then, we apply the simple random sampling for each group. For each group, we randomly select only 15 employees. The final data sample consists of 30 employees and half of them are female.
- This sampling technique is usually applied when we have some constraints for the resulting data sample and the population distribution is skewed.

Cluster sampling



- Step 1: define the population.
- Step 2: divide data elements in the population into clusters.
  - Each cluster should be as diverse as possible; every characteristic of the entire population to be represented in each cluster.
  - Each cluster should have a similar distribution of characteristics as the distribution of the population as a whole.
  - Taken together, the clusters should cover the entire population; and no overlapping.

Cluster sampling (cont'd)

- Step 3: randomly select clusters to use as your sample.
  - If each cluster is itself a mini-representation of the larger population, randomly selecting and sampling from the clusters allows you to imitate simple random sampling, which in turn supports the validity of your results.
  - Conversely, if the clusters are not representative, then random sampling will allow you to gather data on a diverse array of clusters, which should still provide you with an overview of the population as a whole.
- Step 4: collect all the data elements in each selected clusters to form the resulting data sample.

Multi–stage cluster sampling

- In multistage cluster sampling, rather than collect data from every single element in the selected clusters, we randomly select individual elements from within the cluster to form the final data sample. This is called **double–stage cluster sampling**.
- We can also continue this procedure, taking progressively smaller and smaller random samples, which is usually called **multi–stage cluster sampling**.

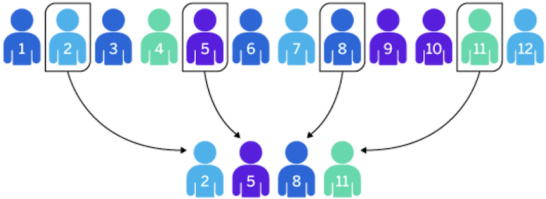
Cluster sampling: advantages and disadvantages

- Advantages:
  - Cluster sampling is time– and cost–efficient, especially for samples that are widely geographically spread and would be difficult to properly sample otherwise.
  - Because cluster sampling uses randomization, if the population is clustered properly, your study will have high external validity because your sample will reflect the characteristics of the larger population.
- Disadvantages:
  - Internal validity is less strong than with simple random sampling, particularly as you use more stages of clustering.
  - If your clusters are not a good mini–representation of the population as a whole, then it is more difficult to rely upon your sample to provide valid results.
  - Cluster sampling is much more complex to plan than other forms of sampling.

Systematic sampling

- In this technique, only the first element is selected randomly. From the second, the selection is based on the previous one.
- First, all the elements in the population are placed in a sequence in random order.
- Suppose the population size is N and the size of the resulting sample is n (normally $n \ll N$). Then divide the population sequence into n sub-sequences with length $k = \lfloor \frac{N}{n} \rfloor$ (except the length of the last sub-sequence may be greater than k).
- The sampling starting by randomly selecting the first element from the first sub-sequence, say at the position/index i_1. Then, the position of the second element is $i_2 = i_1 + k$, the third element is at $i_3 = i_2 + k = i_1 + 2k$. Similarly, $i_n = i_{n-1} + k = i_1 + (n-1)k$.
- For example, with $N = 22$, $n = 5$, then $k = \lfloor \frac{22}{5} \rfloor = 4$ (the last sub-sequence has length of 6). Suppose the first element is selected at $i_1 = 2$, then the indexes of all the other elements of the sample are 6, 10, 14, and 18.

Systematic sampling example



The above figure shows an example of systematic sampling with:

- $N = 12$
- $n = 4$
- $k = 3$
- The first element is randomly selected at position 2, the subsequent selections are at 5, 8, and 11 (with step $k = 3$).

Multi–stage sampling

- Multi–stage sampling is often considered an extended version of cluster sampling.
- In multi–stage sampling, we divide the population into clusters and select some clusters at the first stage. At each subsequent stage, you further divide up those selected clusters into smaller clusters, and repeat the process until you get to the last step. At the last step, you only select some members of each cluster for your sample.
- Like in single–stage sampling, we start by defining the target population. But in multi–stage sampling, we do not need a sampling frame that lists every member of the population. That is why this method is useful for collecting data from large, dispersed populations.

Multi–stage sampling example

- In order to evaluate the knowledge of high–school students national-wide and independent with the national high–school graduation examination, The Ministry of Education and Training (MOET) want to organize an interview with 10,000 students from different regions of the country.
- MOET could not interview students from all high schools from all districts/cities/provinces due to the time, human resource, and cost constraints. MOET will perform a multi–stage sampling as follows:
- **Step 1**: all provinces and cities are put into different clusters based on the geographical regions. Clusters are Northwest, Northeast, Red River Delta, North Central Coast, South Central Coast, Central Highlands, Southeast, and Mekong River Delta. From each region, several provinces and cities will be selected randomly. For example:
  - Son La, Lai Chau, Ha Giang from Northwest
  - Hanoi, Hai Duong from Red River Delta
  - ...
  - Bac Lieu, Can Tho, Tra Vinh from Mekong River Delta

Multi–stage sampling example (cont'd)

- **Step 2**: All high schools from the selected provinces/cities above will be classified into different sub–clusters like region Khu vuc 1 (KV1), Khu vuc 2 (KV2), Khu vuc 2 nong thon (KV2-NT), and Khu vuc 3 (KV3). From each of these clusters, MOET randomly selects 25 high schools and the to get a final total 100 high schools. These 100 high schools represent for all geographical regions and they can be at the city, town, country side, or mountainous areas.
- **Step 3**: From each of 100 selected high schools, select 100 12th–grade students in order to form the resulting data sample with 10,000 students.
- The final data sample of 10,000 students are very diverse, including students from different regions of the country. They can live at city, town, country side, or they can be ethnic minority. This is obviously a suitable sample for MOET to evaluate the high–school education quality. In reality, MOET can even perform a more complicated sampling to meet other constraints that they may have.

Non–random sampling methods

Non–random sampling (or non–probability sampling) is a sampling approach where the selection of elements from a population is not based on the randomness. Rather, the selection is based on the human understanding of the population. The resulting data sample, therefore, can be biased. Here are some well–known non–random sampling techniques:

1. Convenience sampling
2. Judgement sampling
3. Quota sampling
4. Snowball sampling

Convenience sampling

- Convenience sampling is defined as a method adopted by researchers where they collect market research data from a conveniently available pool of respondents.
- It is commonly used sampling technique as it is incredibly prompt, uncomplicated, and economical. In many cases, members are readily approachable to be a part of the sample.
- In most cases, testing the entire community is practically impossible because they are not easy to reach. Researchers also use convenience sampling in situations where additional inputs are not necessary. There are no criteria required to be a part of this sample. Thus, it becomes incredibly simplified to include elements in this sample. All components of the population are eligible and dependent on the researcher's proximity to get involved in the sample.
- The researcher chooses members merely based on proximity and doesn't consider whether they represent the entire population or not. Using this technique, they can observe habits, opinions, and viewpoints in the easiest possible manner.

Applications of convenience sampling

- Convenience sampling is applied by brands and organizations to measure their perception of their image in the market. Data is collected from potential customers to understand specific issues or manage opinions of a newly launched product.
- A basic example of a convenience sampling method is when companies distribute their promotional pamphlets and ask questions at a mall or on a crowded street with randomly selected participants.
- There is always a chance that the randomly selected population may not accurately represent the population of interest, thus increasing the chances of bias.

Advantages of convenience sampling

- *Collect data quickly*: in situations where time is a constraint, many researchers choose this method for quick data collection. The rules to gather elements for the sample are least complicated in comparison to techniques such as simple random sampling, stratified sampling, and systematic sampling.
- *Inexpensive to create samples*: the money and time invested in other probability sampling methods are quite large compared to convenience sampling.
- *Easy to do research*: the name of this surveying technique clarifies how samples are formed.
- *Low cost*: low cost is one of the main reasons why researchers adopt this technique.
- *Readily available sample*: Data collection is easy and accessible. Most convenience sampling considers the population at hand.
- *Fewer rules to follow*: tt does not require going through a checklist to filter members of an audience.

Judgement sampling

- Judgmental sampling, also called purposive sampling or authoritative sampling, is a non–probability sampling technique in which the sample members are chosen only on the basis of the researcher's knowledge and judgment.
- As the researcher's knowledge is instrumental in creating a sample in this technique, there are chances that the results will be highly accurate with a minimum margin of error.
- The process of selecting a sample involves the researchers carefully picking and choosing each individual to be a part of the sample. The researcher's knowledge is primary in this sampling process as the members of the sample are not randomly chosen.
- Judgmental sampling is most effective in situations where there are only a restricted number of people in a population who own qualities that a researcher expects from the target population.
- Judgmental sampling is usually used in situations where the target population comprises of highly intellectual individuals who cannot be chosen by using any other probability or non–probability sampling technique.

Advantages of judgement sampling

- *Consumes minimum time for execution*: in this sampling approach, researcher expertise is important and there are no other barriers involved due to which selecting a sample becomes extremely convenient.
- *Allows researchers to approach their target market directly*: there are no criteria involved in selecting a sample except for the researcher's preferences. Due to this, he/she can communicate directly with the target audience of their choice and produce desired results.
- *Almost real–time results*: A quick poll or survey can be conducted with the sample using judgmental sampling since the members of the sample will possess appropriate knowledge and understanding of the subject.

Quota sampling

- In this technique, the population is divided into exclusive sub–groups according to particular criteria. This partitioning can be done at different levels if needed.
- Then, the sampling will be performed for each sub–group using convenience or judgement sampling. And for each sub–group, the selection will have a particular quota.
- For example, the local government want to get feedback from 200 people including 50 old people, 100 employees, and 50 housewives. Then, the local people will be classified into three sub–groups, and then apply convenience or judgement sampling to select desired people with the above quota.

Snowball sampling

- This technique is usually used when we have little information or find it difficult to approach elements in the population.
- Then, if we find the first appropriate element, we will ask that element (normally a person) to introduce more appropriate elements in the population that we did not know or could not approach them directly.
- In this way, the sample will be expanded quickly like a **snowball**.
- For example, if we need to interview a group of LGBT people, but we do not know who they are. We can find the first person to interview and politely ask him/her to introduce other LGBT people in his/her community. Therefore, this technique is also called *referral sampling*.

Outline

1 Introduction

2 Data cleaning

3 Data integration

4 Data transformation

5 Data reduction

6 Data sampling

7 References and Summary

References

[1] J. Han, M. Kamber, and J. Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, Elsevier, 2012 [Book1].

[2] C. Aggarwal. *Data Mining: The Textbook*. Springer, 2015 [Book2].

[3] J. Leskovec, A. Rajaraman, and J. D. Ullman. *Mining of Massive Datasets*. Cambridge University Press, 2014 [Book3].

[4] M. J. Zaki and W. M. Jr. *Data Mining and Analysis: Fundamental Concepts and Algorithms*. Cambridge University Press, 2013 [Book4].

[5] D. Easley and J. Kleinberg. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010 [Book5].

[6] J. VanderPlas. *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly, 2017 [Book6].

[7] J. Grus. *Data Science from Scratch: First Principles with Python*. O'Reilly, 2015 [Book7].

Summary

- Explaining why data preprocessing and preparation is important in data analysis and mining; as well as major tasks in data preprocessing like *data cleaning*, *data integration*, *data transformation*, *data reduction*, etc.
- Data cleaning techniques to handle missing values, incorrect and inconsistent values, as well as deal with noisy data.
- Introducing several concepts in data integration such as entity identification, redundancy and correlation analysis, data tuple duplication, etc.
- Important concepts and techniques in data transformation with data conversion, data discretization, data smoothing, data aggregation, data scaling and normalization, etc.
- Important concepts and techniques in data reduction such as PCA, attribute subset selection, histograms, clustering, data cube aggregation, as well as various random and non–random data sampling methods.